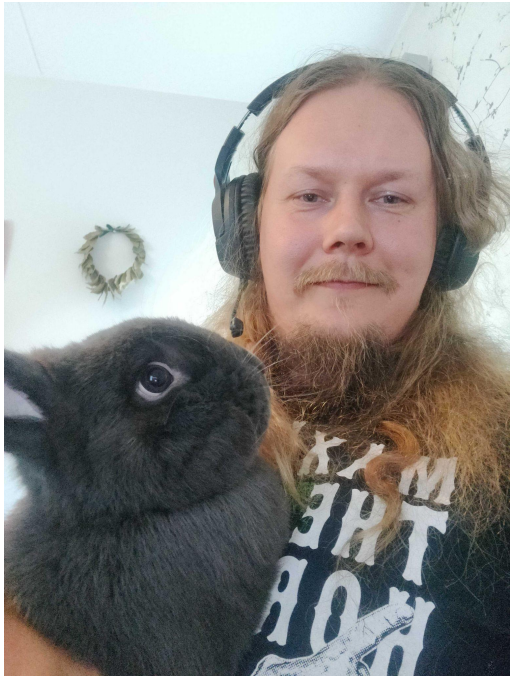
Terraform blocks you don’t know

Lauri Suomalainen 2026

Who is this Lauri Suomalainen person?

- Native of Finland
- Master of Computers and Science
- Wrangler of Clouds
- Friend of developers
- Enjoyer of music
- Climber of rocks
- Player of games
- Servant of rabbits

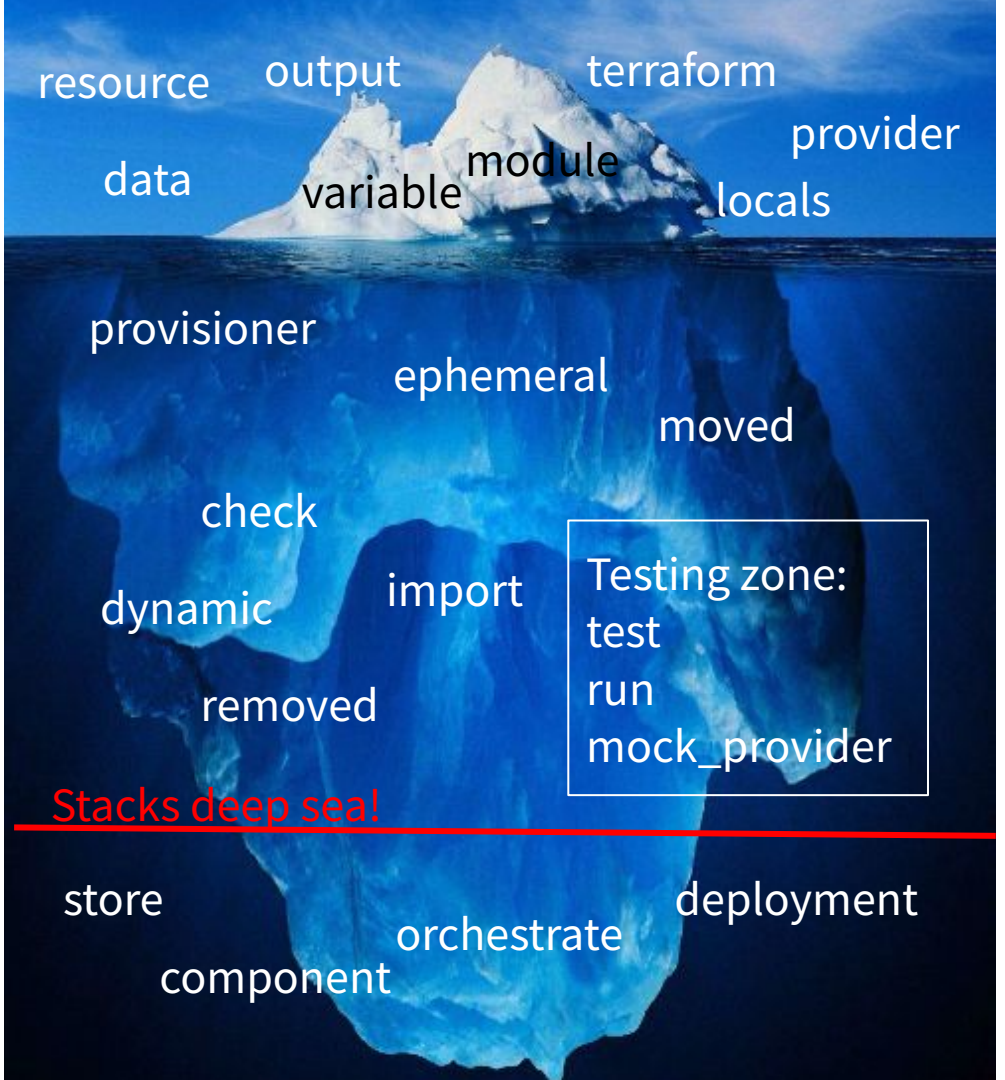


Motivation

- If I don’t know, there’s a good chance I’m not alone.
- Learning experience
- I generally enjoy obscure things.
- Maybe I find something actually useful?

So many blocks!

We are going for a dive



Provisioner

- Used for things not exactly in the scope of Terraform's declarative model
- In principle: TRY TO AVOID.
  - Use immutable resource definitions, metadata etc.
- Pass data to VMs, use cloudinit etc.

- Tied to resources.
  - To use without a resource, use terraform_data. Replaces null_resource.
- file, local_exec, remote_exec
  - Connections either through ssh or winrm
- Terraform 1.14: Check out out the Action block

```
resource "google_compute_instance" "default" {

/*...*/

metadata = {
    ssh-keys = "terraform:${file("~/.ssh/id_rsa.pub")}"
  }

  provisioner "file" {
    source      = "example.txt"
    destination = "/tmp/example.txt"

    connection {
      type        = "ssh"
      user        = "terraform"
      private_key = file("~/.ssh/id_rsa")
      host        = self.network_interface[0].access_config[0].nat_ip
    }
  }
}
```

```
resource "aws_instance" "cluster" {
  count = 3

  # ...
}

resource "terraform_data" "cluster" {
  # Replacement of any instance of the cluster requires re-provisioning
  triggers_replace = aws_instance.cluster[*].id

  # Bootstrap script can run on any instance of the cluster
  # So we just choose the first in this case
  connection {
    host = aws_instance.cluster[0].public_ip
  }

  provisioner "remote-exec" {
    # Bootstrap script called with private_ip of each node in the cluster
    inline = [
      "bootstrap-cluster.sh ${join(" ", aws_instance.cluster[*].private_ip)}",
    ]
  }
}
```

Ephemeral

- Used to create temporary resources
- Not stored in the state, don’t show up in plan files.

- Obviously, creating randomised resources is not very idempotent.
- Best use case is to handle sensitive values from a KMS and prevent them from leaking into plaintext output.

```
ephemeral "random_password" "db_password" {
  length           = 16
  override_special = "!#$%&*()-_=+[]{}<>:?"
}

resource "google_secret_manager_secret" "db_password" {
  secret_id = "db-password"
  replication {
    auto {}
  }
}

resource "google_secret_manager_secret_version" "db_password" {
  secret      = google_secret_manager_secret.db_password.id
  secret_data_wo = ephemeral.random_password.db_password.result
  secret_data_wo_version = 1
}
```

```
ephemeral "google_secret_manager_secret_version" "db_password_access" {
  secret      = google_secret_manager_secret.db_password.id
  version     = "latest"
  depends_on = [google_secret_manager_secret_version.db_password]
}

resource "google_sql_database_instance" "example" {
  /*...*/
}

resource "google_sql_user" "example" {
  name     = "example"
  instance = google_sql_database_instance.example.name
  password_wo =
ephemeral.google_secret_manager_secret_version.db_password_access.secret_data
  password_wo_version =
google_secret_manager_secret_version.db_password.secret_data_wo_version
}
```

Check

- Check on conditions outside of usual resource lifecycle
- Failing does not block plans or applies.

- Could be used, if your resources rely on external services
  - Check if endpoint configurations are valid
  - Check if datasources return correct data.
  - Shoutout to Devops Rob and terracurl.
- You can use postconditions, if you require operations to be blocked on failure

```
resource "google_compute_instance" "datadog_vm" {

 /* ... */

  metadata_startup_script = <<-EOT
    #!/bin/bash
    DD_API_KEY=${var.datadog_api_key}
    DD_AGENT_MAJOR_VERSION=7
    DD_SITE="datadoghq.com"
    apt-get update
    apt-get install -y curl gnupg
    curl -s https://install.datadoghq.com/scripts/install_script_agent7.sh | bash -s --
--api-key $DD_API_KEY --site $DD_SITE
  EOT
}
```

```
check "datadog_endpoint" {
  data "http" "datadog_validate" {
    url = "https://api.datadoghq.com/api/v1/validate?api_key=${var.datadog_api_key}"
  }
  assert {
    condition     =
trimspace(jsondecode(data.http.datadog_validate.response_body)["valid"]) == true
    error_message = "Datadog API key is invalid or endpoint is unreachable."
  }
}
```

dynamic

- Create multiple nested blocks in a resource
- Could even be used to create blocks conditionally.
- Works like a for block, but a bit neater
- Can even be nested within themselves

- A general rule with dynamic terraform applies here too: Use sparsely.
  - Overuse debilitates readability, makes troubleshooting a pain and is often a symptom of flawed design.

```
resource "google_compute_firewall" "example" {

   /* ... */

  dynamic "allow" {
    for_each = var.allow_rules
    content {
      protocol = allow.value.protocol
      ports    = allow.value.ports
    }
  }
}

variable "allow_rules" {
  type = list(object({
    protocol = string
    ports    = list(string)
  }))
  default = [
    { protocol = "tcp", ports = ["22", "80", "443"] },
    { protocol = "icmp", ports = [] }
  ]
}
```

State management blocks - Import

- Don’t do terraform import google_compute_instance.default projects/{{project}}/zones/{{zone}}/instances/{{name}}
- Use the import block!
- Because doing this manually is a pain.
- Explicit on your configuration.
- Generates terraform based on existing resources.
- terraform plan -generate-config-out=generated.tf

```
provider "google" {

}



import {

  id =
"ringed-valor-196605/europe-north1-c/valhei
m-server"

  to =
google_compute_instance.valheim_server

}
```

Check out the new list feature!

- https://developer.hashicorp.com/terraform/language/block/tfquery/list

list block reference

v1.14.x (latest)

The `list` block defines a query that reads existing infrastructure and returns resources that you can import to your Terraform workspace. You can only add `list` blocks to configuration files with `.tfquery.hcl` extensions. This topic provides reference information about configuring `list` blocks in `.tfquery.hcl` files. Refer to Import existing resources in bulk for information about querying and importing resources.

State management blocks - moved

- Rename a resource.

- IMO, of dubious value.

- Could be useful if refactoring a single resource to multiple. See example.

- Adding multiple of these can cause long chains and hinder readability.

- Guess you could use them if you can’t recreate resources.

```
locals {
  instances = tomap({
    big = {
      instance_type = "n1-standard-2"
    }
    small = {
      instance_type = "e2-medium"
    }
  })
}

resource "google_compute_instance" "vm" {
  for_each = local.instances

  name = "example-vm-${each.key}"
  instance_type = each.value.instance_type
  /* .. */
}
moved {
  from = google_compute_instance.vm
  to   = google_compute_instance.vm["big"]
}
```

State management blocks - removed

- Don't do terraform state rm google_compute_instance.default projects/{{project}}/zones/{{zone}}/instances/{{name}}
- You could use removed block?
- You created a resource in terraform but now don't want to manage it programmatically?
- You run a mixture of tf managed and manual resources in the same env. Why?
- Leave an audit trail?
- NOTE: Replace the resource block with removed

```
removed {
  from = google_compute_instance.vm
  lifecycle {
    destroy =false
  }
}
```

Testing blocks

- Terraform unit testing!
  - test: Just used to control test parallelism
  - run: Test actually run in these blocks
  - mock_provider: Can be used to speed up testing: Don’t require credentials.

- Surprisingly nice to sanity check!

- Also surprisingly robust!

- The question is, what should be tested, on what level etc.

```
mock_provider "google" {}

test {
  parallel = true
}

run "set_correct_vm_name" {
  variables {
    vm-name = "test-vm"
  }

  assert {
    condition =
google_compute_instance.test-vm.name ==
"not-test-vm"
    error_message   = "Incorrect vm name"
  }
}
```

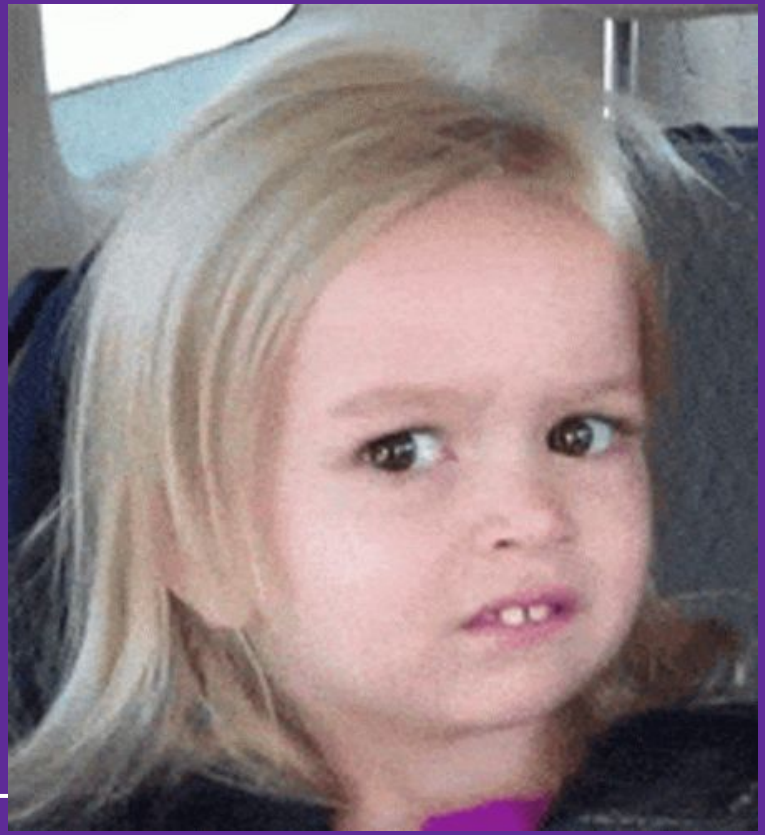
```
run "vm_should_be_in_us" {
  variables {
    vm-name = "test-vm"
  }

  assert {
    condition     =  can(regex("us-*",
google_compute_instance.test-vm.zone))
    error_message = "VM is not in the US"
  }
}
```

My foray into Terraform Stacks

Impressions, complaints and a demo

First impression:

Unimpressed and confused



Impression tally

The good:

- Architecture is clear
- Single convention to reuse modules for multiple environments
- Explicit provider linking & possible to use different providers per resource
- Speculative PR plans are pretty good

The bad:

- Inflexible
- Deviation from standard terraform operation
- HCP only
- Documentation is sparse and lacks essential features
- Out of beta but not much has changed.

Time for the demo!

Featured blocks you don’t know:

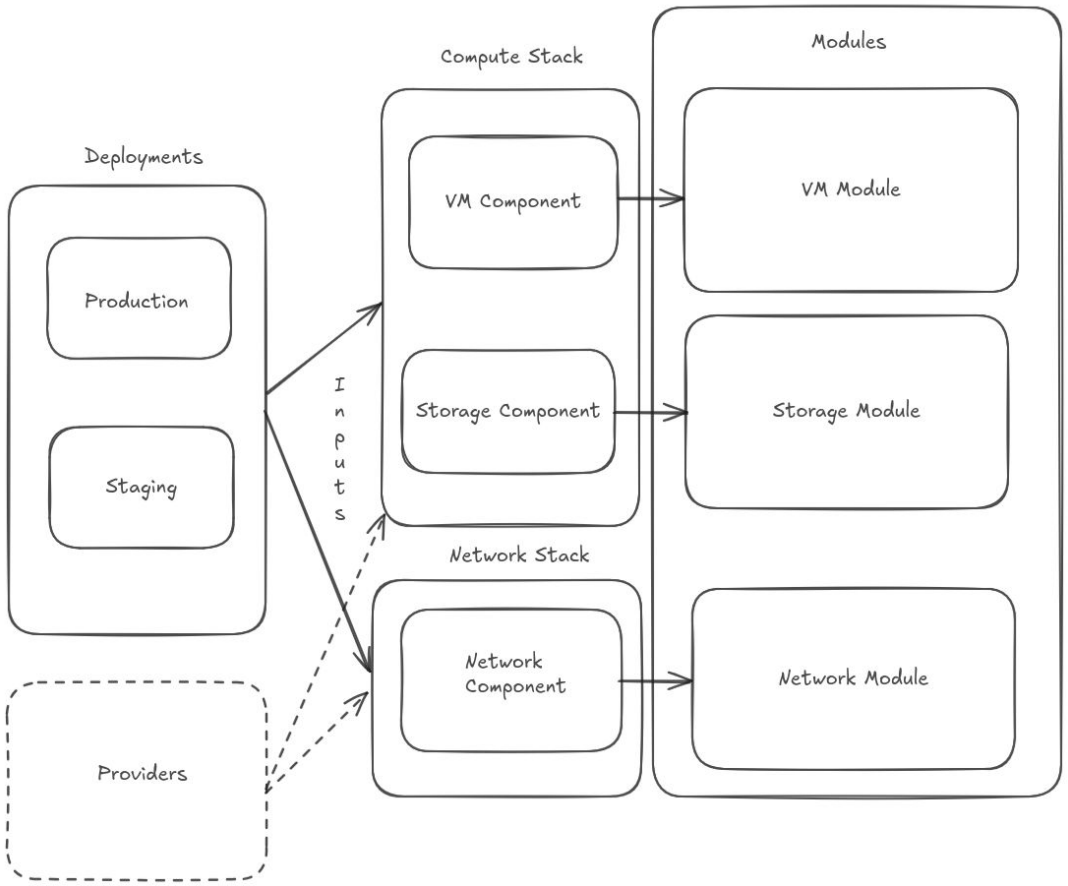
component

deployment

~~orchestrate~~ replaced by deployment_group and deployment_auto_approve

store

Terraform Stacks basic structure



Thank you!

Terraform blocks you don't know
Lauri Suomalainen 2026

Demo repo available in GitHub.

Source:
Terraform official documentation (v1.12.x)
… and some v1.1.4
Yeah, really. I RTFM.